

SOFTWARE PROJECT PRESENTATION

Hospital Management System

A Java & Spring Boot web application for end-to-end hospital operations



PRESENTED BY

Team One

Java 21 · Spring Boot 3.5 · PostgreSQL · Thymeleaf & Bootstrap 5 · Docker

Team One. Members

5 members
Roll numbers as registered



Md. Ismail Hossain
42240302120



Raihan Ahamed
42240302119



Md. Lukman Hossain
42240302090



Mim Akter Laboni
42240302103



Abdullah Al Mamun
42240302123



Built in 11 phases
domain → modules → UI → validation → demo

Project Overview

Layered web application
Controller → Service → Repository → Database

A single web application that replaces a hospital's paper registers. Staff register **patients and doctors**, book **conflict-free appointments**, issue **invoices**, record **payments**, and watch live totals on a **dashboard** — all backed by a PostgreSQL database and protected by validation at every layer.

73

MAIN JAVA CLASSES

6

DOMAIN ENTITIES + 4 ENUMS

5

FUNCTIONAL MODULES

14

AUTOMATED TEST SUITES

The problem

Paper records are slow to search, easy to lose, and let two patients be booked into the same doctor's slot.

Our solution

One browser-based system where every record is searchable, every rule is enforced by the server, and money is never miscalculated.

How it is built

Object-oriented Java with a clean layered architecture, so each module can grow without breaking the others.

Core Modules

Every module: search · filter · paginate
with auto-generated ID codes

P

Patient Management

Register and edit profiles with gender, birth date, blood group and emergency contact. Auto code **PAT-000001**; search by name, phone or code.

D

Doctor Management

Specialization, qualification, consultation fee and availability. Auto code **DOC-000001**; filter by specialization or availability.

A

Appointments

Book a patient with a doctor at a date and time. Status flow **Scheduled** → **Confirmed** → **Completed**, or Cancelled at any point.

B

Billing & Invoices

Consultation, service, medicine and other charges minus discount. Stable **INV-000001** numbers and a print-ready invoice page.

\$

Payments

Cash, card, mobile banking and bank transfer. Partial payments supported; status moves Unpaid → Partially paid → **Paid** automatically.

#

Dashboard

Live counts of patients, doctors, today's and pending visits, plus **total revenue**, **outstanding due** and recent activity.

Beyond the Basics

The engineering that makes it
safe to use for real work

No double booking

Booking locks the patient row and then the doctor row in the database, so two staff members clicking “Save” in the same second can never reserve one slot twice.

No duplicate payments

Each invoice carries a version number. A stale or resubmitted payment form is rejected with a clear “reload and check history” message instead of charging twice.

Validation everywhere

Bean Validation runs in the browser, in the controller and again at the service boundary. Invalid forms keep what the user typed and move focus to the error.

Safe error handling

Friendly 400 / 404 / 405 / 409 / 503 pages. Raw SQL, stack traces and rejected values are never shown to the user; a support reference links to the log.

Responsive, accessible UI

Shared navigation with a mobile off-canvas menu, keyboard focus outlines and confirmation dialogs — and every page still works with JavaScript switched off.

Presentation-ready demo data

One switch seeds 5 doctors, 10 patients, 10 appointments, 5 bills and 4 payments through the real services — and never touches a database that already holds records.

Technology Stack

Server-rendered MVC · no external CDN
all assets served locally

Language & Framework

APPLICATION CORE

Java 21

Spring Boot 3.5.16

Spring MVC

Maven

Jakarta Bean Validation

Data & Persistence

STORAGE LAYER

Spring Data JPA

Hibernate

PostgreSQL 17

BigDecimal money

Row & version locking

Frontend

PRESENTATION LAYER

Thymeleaf

Bootstrap 5.3.8

Bootstrap Icons

Vanilla JavaScript

Print CSS

Testing & Ops

QUALITY & DELIVERY

JUnit 5

Spring Boot Test

H2 test database

Docker & Compose

Spring Actuator

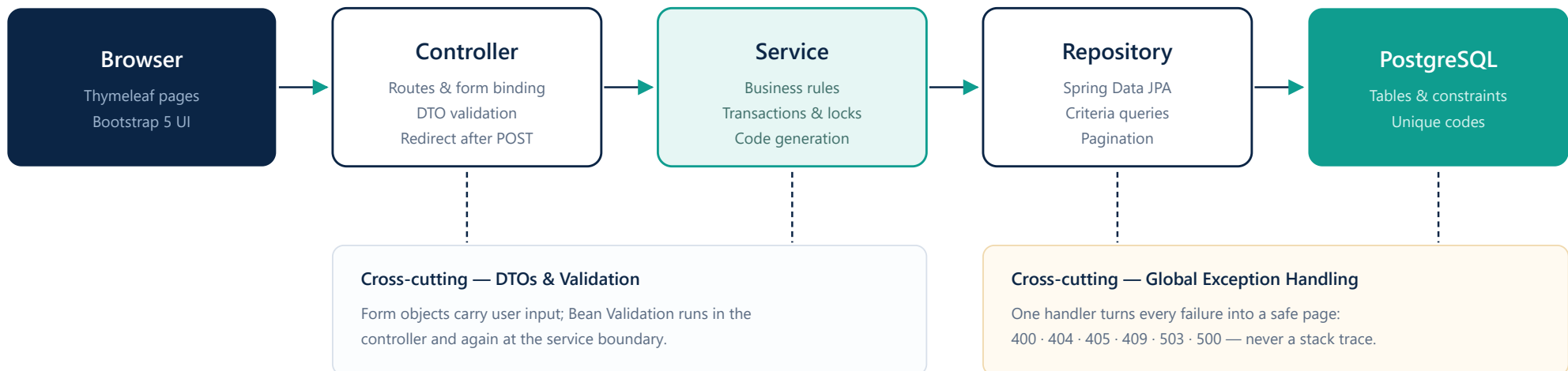
Build & run: multi-stage Docker image — `docker compose up -d` starts the app and the database together.

Config: environment variables for database credentials, hospital timezone and optional demo data.

Health: Actuator endpoints report application and database status at any time.

Application Architecture

Each layer has one job
and talks only to its neighbour

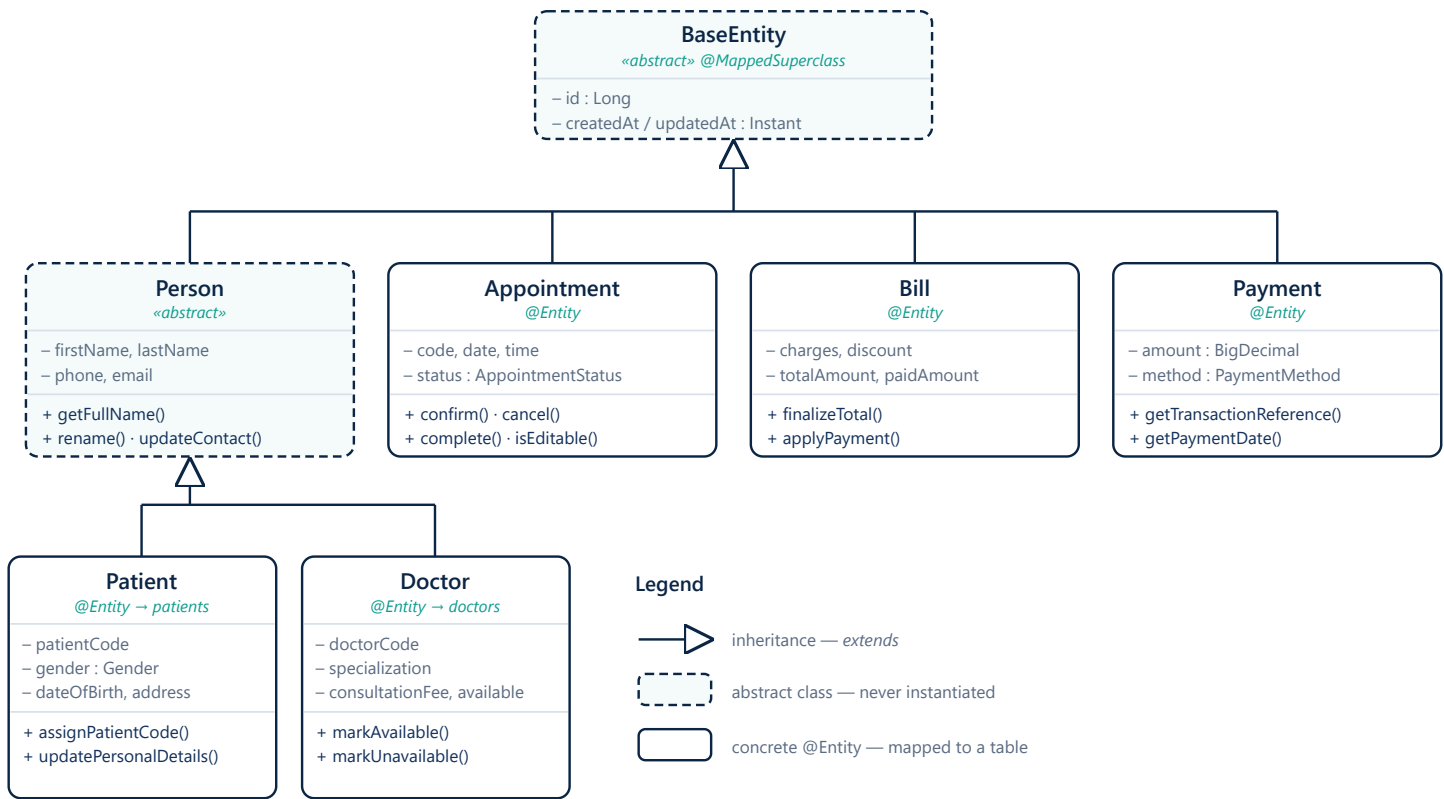


Example — what happens when staff book an appointment

1. The booking form is posted to **AppointmentController**, which binds and validates the DTO.
2. **AppointmentService** opens a transaction and locks the patient row, then the doctor row.
3. The slot is re-checked under that lock — a clash returns a field error, never a saved conflict.
4. The repository saves the row, the code **APT-000001** is derived from its ID,
5. and the browser is redirected to the appointment page — so a refresh never books twice.

Class Diagram

Inheritance hierarchy of the domain model
package com.example.hms.domain



RELATIONSHIPS

- Appointment → Patient many-to-one
- Appointment → Doctor many-to-one
- Bill → Patient many-to-one
- Bill → Appointment one-to-one
- Payment → Bill many-to-one

ENUMERATIONS

- Gender — MALE, FEMALE, OTHER...
- AppointmentStatus — SCHEDULED, CONFIRMED, COMPLETED, CANCELLED
- PaymentStatus — UNPAID, PARTIALLY_PAID, PAID
- PaymentMethod — CASH, CARD, MOBILE, BANK

WHY IT MATTERS

ID and audit timestamps are written **once** in BaseEntity and inherited by every entity.

The Four Pillars in Our Code

Not theory — each pillar points
at a real class in the project

1

Encapsulation

Every entity field is **private**. State changes only through guarded methods such as `rename()` and `applyPayment()`, which validate before assigning. IDs and audit timestamps have no setters at all.

2

Inheritance

`Patient` and `Doctor` extend `Person`, which extends `BaseEntity`. Name, contact, ID and timestamp behaviour is written once and reused by all six entities.

3

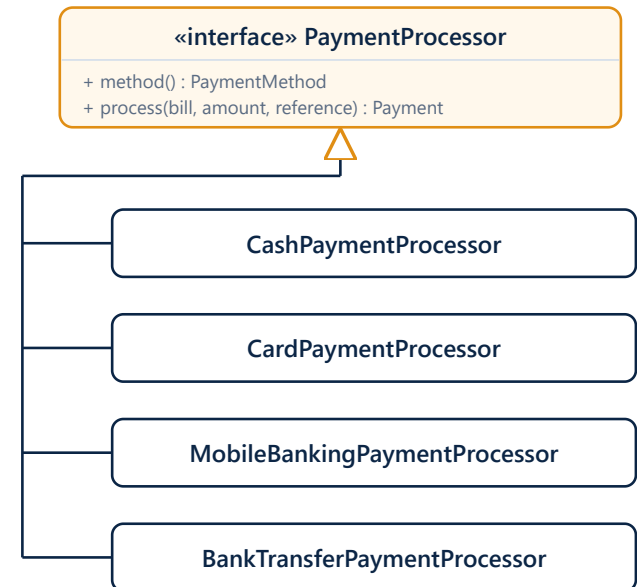
Polymorphism

Four `PaymentProcessor` implementations are injected together. The billing service calls `process()` without knowing whether the money arrived as cash, card, mobile banking or bank transfer.

4

Abstraction

`BillingCalculator` and `PaymentProcessor` are interfaces; `BaseEntity` and `Person` are abstract. Services depend on the contract, never on a concrete class.



One contract · four interchangeable implementations

The payoff

Adding a fifth payment method means writing **one new class** that implements `PaymentProcessor` — no existing service, controller or template has to change. That is the Open/Closed principle in practice.

Benefits

What the hospital gains
and what we gained as a team

01**Paperless, instantly searchable records**

Find any patient or doctor by name, phone or code in seconds instead of turning pages in a register.

02**Zero double bookings**

Database-level locking guarantees a doctor — and a patient — can hold only one appointment in a given slot.

03**Accurate money, every time**

`BigDecimal` arithmetic, validated discounts and derived totals: no rounding loss and no accepted overpayment.

04**Real-time financial insight**

Revenue collected and outstanding dues appear on the dashboard the moment a payment is recorded.

05**Trustworthy data**

Validation in the browser, controller, service and database, plus creation and update timestamps on every record.

06**Runs anywhere in minutes**

One `docker compose up` starts the application and PostgreSQL — no manual setup on the demo machine.

07**Easy to extend**

The layered OOP design lets a new module — pharmacy, lab reports, wards — plug in without touching existing code.

08**Learning outcome for Team One**

Hands-on practice with OOP principles, Spring Boot, relational modelling, concurrency control and automated testing.

Thank You

Questions & live demonstration

Team One

Md. Ismail Hossain · Raihan Ahamed · Md. Lukman Hossain
Mim Akter Laboni · Abdullah Al Mamun